

Video Diffusion Models

Complete Technical Analysis

for From-Scratch Implementation

Based on: Ho et al. "Video Diffusion Models" (arXiv:2204.03458)

Google Research, April 2022

Abstract

This document provides a comprehensive technical analysis of the Video Diffusion Models paper, the first work to successfully apply diffusion models to video generation. We cover mathematical foundations, architectural details, training procedures, and sampling algorithms for from-scratch implementation.

Document Version: 1.0 | Last Updated: 2024

Table of Contents

1	Introduction	3
1.1	Motivation	3
1.2	Key Contributions	3
2	Diffusion Model Background	4
2.1	Forward Process	4
2.2	Training Objective	4
2.3	Classifier-Free Guidance	4
3	3D U-Net Architecture	5
3.1	Overview	5
3.2	Space-Time Factorization	5
3.3	Architecture Details	5
4	Training Methodology	6
4.1	Cosine Noise Schedule	6
4.2	Joint Image-Video Training	6
4.3	Hyperparameters	6
5	Sampling & Reconstruction Guidance	7
5.1	Standard Ancestral Sampling	7
5.2	Reconstruction Guidance (Key Innovation)	7
5.3	Applications	7
6	Experiments	8
6.1	UCF101 (Unconditional Generation)	8
6.2	Video Prediction Results	8
6.3	Reconstruction Guidance vs Replacement	8
7	Implementation Guide	9
7.1	Core Components	9
8	References	10

1 Introduction

1.1 Motivation

Video generation is one of the most challenging tasks in generative modeling, requiring both **spatial coherence** (realistic frames) and **temporal coherence** (plausible motion).

Challenge: Memory Requirements

A 16-frame video at 64×64 resolution:

$$\text{Data} = 16 \times 64 \times 64 \times 3 = 196,608 \text{ values} \quad (1)$$

This is **16× more** than a single image!

1.2 Key Contributions

#	Contribution
1	First video diffusion model with 3D U-Net
2	Space-time factorized attention (1000× speedup)
3	Joint image-video training (3.4× quality improvement)
4	Reconstruction guidance for video extension
5	State-of-the-art on UCF101, BAIR, Kinetics-600

Table 1: Summary of paper contributions

2 Diffusion Model Background

2.1 Forward Process

Definition 2.1.1 (Forward Process): The forward process gradually adds noise to data $\mathbf{x}$:

$$q(\mathbf{z}_t \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}_t; \alpha_t \mathbf{x}, \sigma_t^2 \mathbf{I}) \quad (2)$$

where α_t, σ_t define a noise schedule with decreasing log-SNR $\lambda_t = \log\left(\frac{\alpha_t^2}{\sigma_t^2}\right)$.

Theorem 2.1.1 (Markov Property):

$$q(\mathbf{z}_t \mid \mathbf{z}_s) = \mathcal{N}\left(\mathbf{z}_t; \left(\frac{\alpha_t}{\alpha_s}\right) \mathbf{z}_s, \sigma_{t|s}^2 \mathbf{I}\right) \quad (3)$$

where $\sigma_{t|s}^2 = (1 - e^{\lambda_t - \lambda_s}) \sigma_t^2$ for $s < t$.

2.2 Training Objective

Definition 2.2.1 (ε -Prediction Loss):

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}, \varepsilon, t} [\| \varepsilon_\theta(\mathbf{z}_t) - \varepsilon \|_2^2] \quad (4)$$

where $\mathbf{z}_t = \alpha_t \mathbf{x} + \sigma_t \varepsilon$ and $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$.

2.3 Classifier-Free Guidance

Definition 2.3.1 (Guidance Formula):

$$\tilde{\varepsilon}_\theta(\mathbf{z}_t, \mathbf{c}) = (1 + w) \varepsilon_\theta(\mathbf{z}_t, \mathbf{c}) - w \varepsilon_\theta(\mathbf{z}_t) \quad (5)$$

Higher $w \rightarrow$ better quality, lower diversity.

3 3D U-Net Architecture

3.1 Overview

Model Input/Output

Input: Noisy video $z_t \in \mathbb{R}^{T \times H \times W \times C}$, conditioning c , timestep t

Output: Predicted noise $\varepsilon_\theta \in \mathbb{R}^{T \times H \times W \times C}$

3.2 Space-Time Factorization

The key innovation is **factorizing** spatial and temporal processing:

Block Structure:

```

Input:  $x \in \mathbb{R}^{(T \times H \times W \times C)}$ 
  ↓
Spatial Conv (1×3×3) - process each frame
  ↓
Spatial Attention - attend over (H×W)
  ↓
Temporal Attention - attend over T
+ Relative position embeddings
  ↓
Output:  $y \in \mathbb{R}^{(T \times H \times W \times C')}$ 

```

Computational Advantage

Full 3D: $O(T^2 H^2 W^2)$

Factorized: $O(T H^2 W^2 + H W T^2)$

For $T = 16, H = W = 64$: **1000× reduction!**

3.3 Architecture Details

Stage	Resolution	Channels	Shape
Input	64×64	3	16×64×64×3
Enc Block 1	32×32	256	16×32×32×256
Enc Block 2	16×16	512	16×16×16×512
Enc Block 3	8×8	1024	16×8×8×1024
Bottleneck	4×4	2048	16×4×4×2048
Dec + Skip	...	...	Symmetric
Output	64×64	3	16×64×64×3

Table 2: U-Net tensor dimensions

4 Training Methodology

4.1 Cosine Noise Schedule

Definition 4.1.1 (Cosine Schedule):

$$\alpha_t^2 = \cos^2\left(\frac{t+s}{1+s} \cdot \frac{\pi}{2}\right) \quad (6)$$

where $s = 0.008$ is a small offset. Log-SNR clipped to $[-20, 20]$.

4.2 Joint Image-Video Training

Key Result

Adding 8 independent images per video batch:

- **FVD**: 205 $\rightarrow$ 61 (3.4 $\times$ improvement)
- **FID**: 37 $\rightarrow$ 15 (2.5 $\times$ improvement)

The attention mask ensures video frames interact with each other, while images only attend to themselves.

4.3 Hyperparameters

Param	UCF101	BAIR	Kinetics	Text2Video
Channels	256	128	256	256
Mult	1,2,4,8	1,2,3,4	1,2,4,8	1,2,4,8
LR	3e-4	2e-4	2e-4	3e-4
Steps	60K	660K	220K	700K
Hardware	128 TPU	128 TPU	256 TPU	128 TPU

Table 3: Training configurations

5 Sampling & Reconstruction Guidance

5.1 Standard Ancestral Sampling

Starting from $z_1 \sim \mathcal{N}(0, I)$:

$$z_s = \tilde{\mu}_{s|t}(z_t, \hat{x}_\theta) + \tilde{\sigma}_{s|t}\epsilon \quad (7)$$

5.2 Reconstruction Guidance (Key Innovation)

Theorem 5.2.1 (Reconstruction Guidance): For conditional sampling $p(\mathbf{x}^b \mid \mathbf{x}^a)$:

$$\tilde{\mathbf{x}}_\theta^b = \hat{\mathbf{x}}_\theta^b - \frac{w_r \alpha_t}{2} \nabla_{\mathbf{z}_t^b} \|\mathbf{x}^a - \hat{\mathbf{x}}_\theta^a\|_2^2 \quad (8)$$

Why is this needed?

The naive replacement method gives $\mathbb{E}[\mathbf{x}^b \mid \mathbf{z}_t]$, but we need $\mathbb{E}[\mathbf{x}^b \mid \mathbf{z}_t, \mathbf{x}^a]$.

Reconstruction guidance adds the missing conditioning gradient!

5.3 Applications

1. **Autoregressive Extension:** Generate 64+ frames from 16-frame model
2. **Temporal Super-Resolution:** Increase frame rate (interpolation)
3. **Spatial Super-Resolution:** $64 \times 64 \rightarrow 128 \times 128$

6 Experiments

6.1 UCF101 (Unconditional Generation)

Method	FID↓	IS↑
MoCoGAN	26998	12.42
TGAN-v2	3431	28.87
DVD-GAN	–	32.97
VideoGPT	–	24.69
Video Diffusion	295	57

Table 4: UCF101: 11× FID improvement, IS approaching real data (60.2)

6.2 Video Prediction Results

Dataset	Prev. SOTA	VDM
BAIR (FVD↓)	86.9	66.9
Kinetics (FVD↓)	25.4	16.2
Kinetics (IS↑)	12.5	15.6

Table 5: Video prediction benchmarks

6.3 Reconstruction Guidance vs Replacement

Method	FVD↓	FID↓	IS↑
Replacement	451	26	7.0
Recon. Guidance	136	14	10.3

Table 6: 3.3× FVD improvement with reconstruction guidance

7 Implementation Guide

7.1 Core Components

7.1.1 1. Noise Schedule

```
class CosineSchedule:
    def __init__(self, s=0.008):
        self.s = s

    def __call__(self, t):
        f = torch.cos((t + self.s) / (1 + self.s) * pi / 2) ** 2
        f0 = math.cos(self.s / (1 + self.s) * pi / 2) ** 2
        alpha = torch.sqrt(f / f0)
        sigma = torch.sqrt(1 - f / f0)
        return alpha, sigma
```

7.1.2 2. Spatial Attention

```
class SpatialAttn(nn.Module):
    def forward(self, x): # (B, C, T, H, W)
        B, C, T, H, W = x.shape
        x = rearrange(x, 'b c t h w -> (b t) (h w) c')
        out = self.attention(x) # Attend over H*W
        return rearrange(out, '(b t) (h w) c -> b c t h w',
                        b=B, t=T, h=H, w=W)
```

7.1.3 3. Temporal Attention

```
class TemporalAttn(nn.Module):
    def forward(self, x): # (B, C, T, H, W)
        B, C, T, H, W = x.shape
        x = rearrange(x, 'b c t h w -> (b h w) t c')
        out = self.attention(x) + self.rel_pos_bias # Over T
        return rearrange(out, '(b h w) t c -> b c t h w',
                        b=B, h=H, w=W)
```

7.1.4 4. Training Step

```
def train_step(model, videos, optimizer):
    t = torch.rand(B)
    alpha, sigma = schedule(t)
    eps = torch.randn_like(videos)
    z_t = alpha * videos + sigma * eps

    eps_pred = model(z_t, t)
    loss = F.mse_loss(eps_pred, eps)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss
```

8 References

1. Ho et al. “Video Diffusion Models.” arXiv:2204.03458, 2022.
2. Ho et al. “Denoising Diffusion Probabilistic Models.” NeurIPS 2020.
3. Nichol & Dhariwal. “Improved DDPM.” ICML 2021.
4. Song et al. “Score-Based Generative Modeling through SDEs.” ICLR 2021.
5. Ho & Salimans. “Classifier-Free Diffusion Guidance.” NeurIPS Workshop 2021.